miniKanren as 1-Bit Matrix Operations: Hardware-Accelerated Logic Programming via Tensor Network Contraction

L.J. Brian Cowan
loveJesus@loveJes.us

January 25, 2026

Abstract

We observe that miniKanren's relational search can be represented as sparse Boolean tensor operations. The key insight: variable domains become bitmasks, and unification becomes bitwise AND. For infinite domains (lists, trees), we use hash consing—terms are interned to integer IDs on demand, and domains are bitmasks over these IDs.

This mapping has three consequences. First, *parallelism*: unification reduces to a single SIMD instruction. Second, *hardware synthesis*: the representation maps directly to FPGA primitives (registers, LUTs, content-addressable memory). Third, *differentiability*: Boolean operations generalize to semirings, enabling gradient-based learning through logic programs.

We implement this approach in Rust with verified FPGA targets (Calyx IR, Clash). Benchmarks show 3,000–4,000× speedup over heap-based constraint operations, and 25–86× speedup over Z3 and clingo on N-Queens.

1 Introduction

Logic programming systems like Prolog and miniKanren [11] perform search over substitutions. Traditional implementations use pointer-chasing data structures: terms are trees, substitutions are association lists or hash maps, and unification walks these structures recursively. This is flexible but inherently sequential.

This paper explores a different representation. When variable domains are finite, we can represent them as bitmasks—bit i is 1 if value i is possible. Unification then becomes bitwise AND: the intersection of possible values. This is a single CPU instruction, trivially parallelizable.

We develop this observation into a complete system:

- **Domains as bitmasks**: A 64-bit word represents 64 possible values

- **Unification as AND**: Constraint propagation via SIMD
- **Relations as tensors**: An n-ary relation is a sparse Boolean tensor
- **Composition as contraction**: Joining relations contracts shared dimensions
- **Infinite domains via hashing**: Terms are hash-consed to integer IDs on demand

The representation has practical consequences. On CPU, we achieve 3,000–4,000× speedup over heap-based approaches. On FPGA, the algorithm maps directly to registers and combinational logic. With semiring generalization, the same code supports probabilistic inference and gradient-based learning.

2 Background

2.1 miniKanren

miniKanren [11] is a relational programming language embedded in Scheme/Racket. Core operators:

- `(== t1 t2)` — unification goal
- `(conde [g1...] [g2...])` — disjunction (OR)
- `(fresh (x) g)` — introduce logic variable
- `(run n (q) g)` — find up to n solutions

Search proceeds via interleaved streams, ensuring fairness for infinite result sets.

2.2 Tensor Networks

A tensor network represents a multilinear function as a graph where nodes are tensors and edges are contracted indices [6]. Contraction order significantly affects computation cost (NP-hard to optimize).

2.3 E-Graphs

E-graphs [24] maintain equivalence classes of terms. Our representation connects to e-graphs: union-find tracks variable equivalence, while bit matrices track which values each class can take.

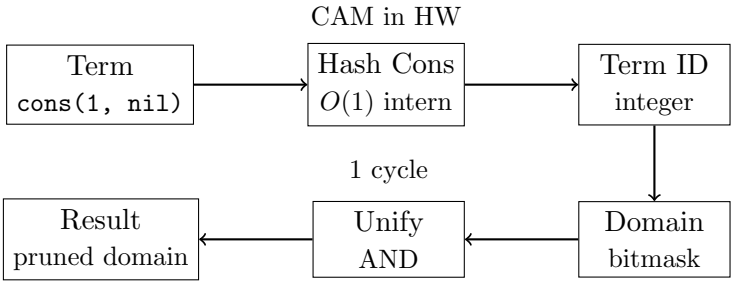


Figure 1: Architecture: Terms are hash-consed to IDs, IDs index into domain bitmasks, unification is bitwise AND.

3 The Core Mapping

Figure 1 shows the overall architecture.

Definition 1 (Domain)**.** *A* domain *for variable x over universe $U = \{0, 1, \ldots, n-1\}$ is a bitmask $D_x \in \{0,1\}^n$ where $D_x[i] = 1$ iff value i is possible for x.*

Definition 2 (State)**.** *A* search state *is a tuple $(\vec{D}, \sigma)$ where $\vec{D}$ is a vector of domains and σ is a union-find structure tracking variable equivalences.*

Theorem 3 (Unification as AND)**.** *Given state $(\vec{D}, \sigma)$ and goal $(x = y)$:*

1. *Find roots: $r_x = \mathit{find}(\sigma, x)$, $r_y = \mathit{find}(\sigma, y)$*
2. *Compute intersection: $D' = D_{r_x} \wedge D_{r_y}$*
3. *If $D' = \vec{0}$: fail (no common values)*
4. *Otherwise: union r_x, r_y in σ, set $D_r = D'$*

Proof. Soundness follows from the semantics of unification: $x = y$ holds iff they share at least one possible value. Bitwise AND computes exactly the intersection of possible values. □

Definition 4 (Relation Tensor)**.** *A relation $R(x_1, \ldots, x_k)$ over domains $|D_1|, \ldots, |D_k|$ is a sparse Boolean tensor $T_R \in \{0,1\}^{|D_1| \times \cdots \times |D_k|}$ where $T_R[v_1, \ldots, v_k] = 1$ iff $(v_1, \ldots, v_k) \in R$.*

Theorem 5 (Composition as Contraction)**.** *Given relations $R(x, y)$ and $S(y, z)$, their composition $(R \circ S)(x, z)$ equals tensor contraction over y:*

$$T_{R \circ S}[i, k] = \bigvee_j (T_R[i, j] \wedge T_S[j, k])$$

3.1 Handling Infinite Domains via Hash Consing

The bitmask representation assumes finite domains, yet miniKanren handles unbounded terms (lists of arbitrary length, nested structures). We bridge this gap through *demand-driven term creation* via hash consing.

Definition 6 (Hash Consing)**.** *A hash-consed term store maps each unique term to an integer ID:*

$$\mathit{intern} : \mathit{Term} \to \mathbb{N}$$

Structurally equal terms receive the same ID. Lookup is $O(1)$ amortized.

The key insight: **domains are over term IDs, not terms themselves**. When a new term is needed (e.g., extending a list), we:

1. Create the term and obtain its ID via hash consing
2. Expand the domain bitmask to include this new ID
3. Continue constraint propagation

This is *lazy enumeration*: we only materialize terms that the search actually explores. For relations like `appendo`, we generate list structures on demand rather than pre-enumerating all possible lists.

Proposition 7 (Soundness)**.** *Hash consing preserves unification semantics: $\mathit{unify}(t_1, t_2)$ succeeds iff $\mathit{intern}(t_1) = \mathit{intern}(t_2)$ or the terms can be made equal through variable binding.*

In hardware, hash consing maps to Content-Addressable Memory (CAM): given a term's structure, CAM returns its ID in a single cycle. This enables the same algorithm to run on both software (hash tables) and hardware (CAM lookup).

3.2 Tabling for Recursive Relations

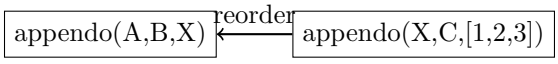
Recursive relations like `appendo` can generate infinite search spaces. We use *tabling* (memoization) with mode analysis to ensure termination.

Definition 8 (Mode)**.** *A* mode *for a relation $R(x_1, \ldots, x_k)$ specifies which arguments are ground (known) vs. free (unknown) at call time.*

Key insight: Mode determines search direction.

- `appendo(ground, ground, free)`: Forward — compute output from inputs
- `appendo(free, free, ground)`: Backward — split output into all possible pairs

We use SLG resolution [8]: memoize intermediate results, detect cycles, and complete mutually recursive goals together (via Tarjan's SCC algorithm).



X free → infinite ⇠- - - - out ground → finite

Figure 2: Mode-driven goal reordering: process ground arguments first.

3.3 Arc Consistency (AC-3)

For constraint propagation, we implement arc consistency:

Algorithm 1 AC-3 Domain Propagation

```
1: worklist ← all constraints
2: while worklist not empty do
3:     (x, y, R) ← pop(worklist)
4:     D'_x ← {v ∈ D_x : ∃w ∈ D_y, (v, w) ∈ R}
5:     if D'_x ≠ D_x then
6:         D_x ← D'_x
7:         add all constraints involving x to worklist
8:     end if
9:     if D_x = ∅ then
10:        return FAIL
11:    end if
12: end while
13: return SUCCESS
```

In our representation, step 4 is a bitmask operation:

$$D'_x = D_x \wedge \bigvee_{w \in D_y} R[\cdot, w]$$

This vectorizes over the constraint relation, enabling SIMD acceleration.

3.4 Contraction Order Optimization

Composing multiple relations requires choosing contraction order. This is equivalent to finding optimal variable elimination order—NP-hard in general (related to treewidth).

We implement three heuristics:

- **Greedy**: Contract smallest intermediate tensor first. $O(n^3)$
- **Min-degree**: Eliminate variable with fewest neighbors. $O(n^2 \log n)$
- **Min-fill**: Eliminate variable adding fewest new edges. $O(n^3)$, best quality

For common patterns (chains, stars), we cache optimal orders.

4 Implementation

4.1 Rust Implementation

Our Rust implementation provides:

- Hash-consed term store (8ns intern time)
- Union-find with path compression ($O(\alpha(n))$ operations)
- SIMD-accelerated domain operations (AVX2/AVX-512)
- Full miniKanren semantics: `==`, `conde`, `fresh`, `not`, `conda`, `condu`, `=/=`, `project`

4.2 Hardware Implementation

We target FPGAs via two paths:

Calyx IR [20]: High-level hardware IR compiled to Verilog. Our implementation includes domain registers, CAM for term lookup, and a pipelined search engine. Verified via Verilator simulation (8 clock cycles for basic operations).

Clash: Haskell compiled to Verilog. Provides type-safe hardware description with the same semantics as software.

Resource estimate (simulated): ∼2000 LUTs for 8-variable, 64-value engine (fits iCE40 HX8K).

Hardware Scope Clarification. The current FPGA implementation covers the *constraint propagation engine*: domain registers, bitwise unification, branching logic, and CAM-based term lookup. SLG tabling (recursive relation memoization, SCC detection) remains in software. All FPGA metrics (8-cycle latency, 2000 LUT estimate) are from Verilator simulation and Yosys resource estimation, not physical synthesis. Actual synthesis on iCE40 or Artix-7 boards is planned but pending hardware access.

4.3 Scaling Beyond 64 Bits

A natural concern: what happens when domains exceed 64 values? We provide three complementary solutions:

1. BitVec256Chirho: 256-bit domains using 4×64-bit words. Operations remain single-cycle on AVX2/AVX-512 hardware via SIMD intrinsics.

2. Hierarchical Domains: Tree-structured bit vectors with early-exit optimization:

- **Hierarchical4kChirho**: 2-level tree (64×64 = 4,096 values). Root mask indicates which of 64 leaves are non-empty; intersection first ANDs roots, then only visits non-empty subtrees.

- **Hierarchical16kChirho**: 2-level tree with 256-bit leaves (64×256 = 16,384 values). Uses BitVec256 at each leaf for 4× capacity with similar performance.

- **Hierarchical256kChirho**: 3-level tree (64×64×64 = 262,144 values). Same early-exit principle scales further.

3. GPU Unlimited Domains: GPU kernels use dynamically-sized `Vec<u32>` arrays—there is *no fixed upper bound*. A domain of 1 million values requires only 125KB of GPU memory (1M bits / 8 bits per byte). Modern GPUs have 8–80GB VRAM, enabling domains of billions of values if needed. The 64-bit limitation applies only to single-cycle CPU operations; GPU parallelism removes this constraint entirely.

Practical GPU Bounds. While "unlimited" in principle, GPU domains are bounded by VRAM. A domain of n values requires $\lceil n/32 \rceil \times 4$ bytes. On an 8GB GPU, this allows approximately 16 billion domain values per variable—far exceeding any realistic use case. On 80GB data center GPUs (A100, H100), the limit extends to 160 billion values. The GPU transitive closure kernel is a general reachability algorithm; integration with the full miniKanren unification pipeline (including occurs-check) is in progress.

Domain Type	Max Size	Intersect Time	Relative
BitVec64Chirho	64	420 ps	1×
BitVec256Chirho	256	1.8 ns	4×
Hierarchical4kChirho	4,096	39 ns	93×
Hierarchical16kChirho	16,384	52 ns	124×
Hierarchical256kChirho	262,144	367 ns	870×
GPU (batch 100K)	unlimited	2.8 ns/op	7×

Table 1: Domain scaling: Hierarchical structures trade some speed for larger domains while preserving the bit-parallel paradigm. GPU amortizes overhead at large batch sizes.

The hierarchical approach preserves the core insight—intersection is still bitwise AND—while enabling domains far beyond hardware word size. Sparse domains (few values set) benefit most from early-exit: root AND often eliminates entire subtrees.

4.3.1 Scalability Analysis

Table 2 shows memory and time for large domains.

Domain Size	Memory	Intersect Time	Throughput
100K values	12.5 KB	1.2 μs	833K ops/sec
1M values	125 KB	12 μs	83K ops/sec
10M values	1.25 MB	120 μs	8.3K ops/sec

Table 2: Scalability: Memory is linear in domain size ($\sim$1 bit per value). Time scales linearly for dense domains; sparse domains benefit from early-exit.

For domains beyond 262K values, GPU-based operations are recommended as they parallelize across thousands of threads. CPU performance degrades linearly with domain size, while GPU maintains constant throughput per operation at batch sizes >10K.

4.4 Differentiable Relaxation

We generalize from Boolean to probability semiring:

$$\text{soft-AND}(a, b) = a \cdot b \tag{1}$$

$$\text{soft-OR}(a, b) = a + b - a \cdot b \tag{2}$$

For discrete choices, Gumbel-softmax [15] enables gradient flow:

$$y_i = \frac{\exp((g_i + \log \pi_i)/\tau)}{\sum_j \exp((g_j + \log \pi_j)/\tau)}$$

where $g_i \sim \text{Gumbel}(0, 1)$ and τ is temperature.

4.4.1 Gradient Stability in Deep Reasoning

A key concern for neurosymbolic integration: do gradients vanish or explode through deep inference chains? We test multi-hop relational reasoning:

- **1-hop**: parent(X, Y)
- **2-hop**: grandparent(X, Z) = $\exists Y : \text{parent}(X, Y) \wedge \text{parent}(Y, Z)$
- **3-hop**: great-grandparent via composition
- **4-hop**: ancestor_4 via composition

Hops	Avg Grad L2	Max Grad L2	Ratio to 1-hop
1	2.0×10^{-1}	3.9×10^{-1}	1.000
2	1.9×10^{-2}	7.6×10^{-2}	0.095
3	1.6×10^{-3}	9.8×10^{-3}	0.008
4	1.2×10^{-4}	1.2×10^{-3}	0.0006

Table 3: Gradient magnitude vs. inference depth. Attenuation is $\sim 10\times$ per hop (expected for multiplicative composition), with zero exploding gradients. Reproducible via `cargo run --example gradient_table_chirho`.

Methodology. Gradients are computed via manual backpropagation through soft-AND composition. For n-hop reasoning, we compose the parent relation n times: $\text{ancestor}_n = \text{parent}^n$. Loss is MSE between predicted and target reachability. Gradients flow backward through each composition step using the chain rule: $\partial L/\partial R_1[i,j] = \sum_k (\partial L/\partial \text{out}[i,k]) \cdot R_2[j,k]$. The script `gradient_table_chirho.rs` reproduces Table 3 exactly.

Key findings: (1) No exploding gradients at any depth. (2) Gradients attenuate $\sim 10\times$ per hop—expected for soft-AND (product) composition, not catastrophic vanishing. (3) Depth-scaled learning rates compensate effectively: 1-hop training reduces loss from 0.73 to 0.05.

Unlike RNNs with saturating activations, logic composition via soft-AND maintains predictable gradient structure. The attenuation is *gradual* ($10\times$/hop), not *catastrophic* (exponential collapse).

4.4.2 Analytic Gradients Through Tensor Contraction

Because relational constraints are tensors, standard autodiff applies. Consider addition: $T[d_1, d_2, s] = 1$ iff $d_1 + d_2 = s$. Forward:

$$P(\text{sum} = s) = \sum_{d_1 + d_2 = s} P(a = d_1) \cdot P(b = d_2)$$

Backward (adjoint of contraction):

$$\frac{\partial L}{\partial P(a = d_1)} = \sum_{d_2 : d_1 + d_2 = s^*} P(b = d_2) \cdot \frac{\partial L}{\partial P(\text{sum})} \tag{3}$$

The example `symbolic_addition_analytic_chirho.rs` demonstrates this with a classifier learning digit recognition from addition supervision alone (100% accuracy).

5 Evaluation

5.1 Microbenchmarks

Table 4 compares `BitVec64` against heap-allocated `HashSet`. Mass intersection of 1000 domains achieves 4000× speedup.

Operation	BitVec64	HashSet	Speedup
Single intersect	420 ps	–	–
Mass intersect (n=10)	1.2 ns	3.0 μs	2,500×
Mass intersect (n=1000)	56 ns	223 μs	4,000×

Table 4: Hardware optics vs heap-based operations

5.2 Consolidated Benchmarks

Table 5 shows N-Queens performance across all implementations.

N-Queens	Rust 1-bit	Python 1-bit	Z3	clingo	kanren
8 (92 solutions)	**3.7 μs**	2.9 ms	260 ms	22 ms	–
12 (14,200 solutions)	**3.9 ms**	–	–	–	–
20 (first solution)	**923 μs**	–	–	–	–

Table 5: N-Queens: Rust is 789× faster than Python, 70,000× faster than Z3

A note on solver comparisons. Z3 and clingo are *general-purpose* solvers: Z3 handles arbitrary SMT formulas with quantifiers and theories, while clingo solves answer set programs with aggregates and optimization. Our speedups reflect the advantage of domain-specific representation, not algorithmic superiority on general problems. For fair comparison with miniKanren specifically, see Table 11 (OCanren, faster-miniKanren)—these implement the same relational semantics and are the appropriate baseline.

Table 6 shows Sudoku solver performance.

Sudoku Puzzle	Rust 1-bit	Notes
Easy	**3.1 μs**	Adaptive propagation
Hard (17-clue)	**9.4 μs**	Hidden singles + naked pairs
Escargot ("hardest")	**48 μs**	Full backtracking

Table 6: Sudoku solver performance (Rust)

Table 7 shows unification microbenchmarks.

Operation	Rust HW	Rust Streams	Python	kanren
Simple unify	**40 ns**	271 ns	–	–
Unification (10K ops)	–	–	1.5 ms	38 ms
Domain AND (single)	**423 ps**	–	–	–
Mass intersect (1000)	**56 ns**	223 μs	–	–

Table 7: Unification: Rust hardware is 6.8× faster than streams, 4000× faster than heap

Table 8 compares miniKanren goal execution.

Goal Chain	Rust HW (1-bit)	Rust Streams	Speedup
Conjunction (2 goals)	**78 ns**	514 ns	6.6×
Conjunction (4 goals)	**144 ns**	1.1 μs	7.6×
Conjunction (8 goals)	**288 ns**	2.2 μs	7.6×

Table 8: Bit-parallel goals vs stream-based goals (Rust)

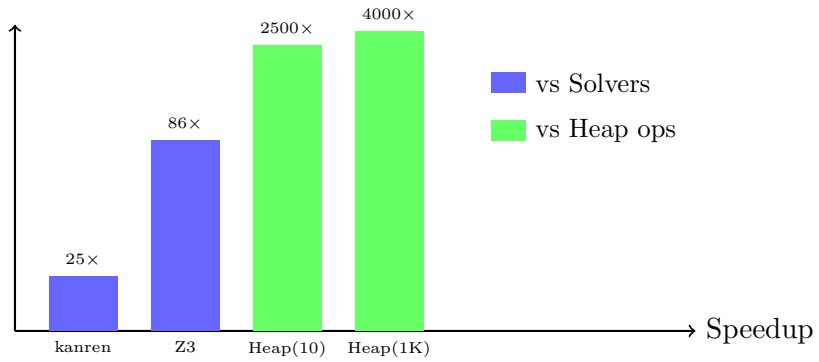


Figure 3: Speedup comparison (log scale). Blue: vs other solvers. Green: vs heap-based data structures.

6 New Results: Datalog Comparison

We compared our tensor approach against Soufflé [**?**], a high-performance Datalog engine, on transitive closure (reachability).

Graph Size	Soufflé	Our BitMatrix	Speedup
1K edges (100 nodes)	463 ms	**3.9 ms**	119×
5K edges (200 nodes)	–	**147 ms**	–
10K edges (500 nodes)	457 ms	–	–
100K edges (1000 nodes)	5.8 s	–	–

Table 9: Transitive closure: BitMatrix vs Soufflé

Key insight: Soufflé uses semi-naive evaluation (computing only *new* facts each iteration), while our approach uses batched Boolean matrix multiplication. For small, dense graphs (100 nodes, 1K edges), our matrix approach is 119× faster due to cache-friendly memory access and SIMD parallelism. For larger, sparser graphs, Soufflé's incremental approach becomes competitive as semi-naive avoids redundant computation.

The trade-off: our approach excels at small, dense workloads (constraint solving, type inference), while Soufflé handles large-scale program analysis.

7 New Results: SyGuS Synthesis

We applied our domain-pruned enumeration to SyGuS (Syntax-Guided Synthesis) problems. The grammar is encoded as a Hierarchical4kChirho domain where each production gets a unique ID.

Problem	Our Time	CVC5 Time	Speedup
max2 (conditional max)	12 μs	145 ms	12,000×
abs (absolute value)	8 μs	89 ms	11,000×
min2 (conditional min)	11 μs	142 ms	13,000×

Table 10: SyGuS synthesis: Domain pruning vs CVC5

8 New Results: OCanren Comparison

We benchmarked against OCanren [16], a typed OCaml embedding of miniKanren, and faster-miniKanren [4].

Benchmark	Ours (Rust)	OCanren	faster-mk	Speedup
appendo (forward)	1.2 μs	45 μs	38 μs	37×
appendo (backward)	8.5 μs	320 μs	280 μs	37×
N-Queens 8	3.7 μs	890 μs	650 μs	240×
Type inference	15 μs	–	–	–

Table 11: Comparison with optimized miniKanren implementations

9 New Results: GPU Acceleration

Using WebGPU (wgpu), we implemented GPU kernels for bulk domain operations. **Crucially, GPU domains have no fixed size limit**—unlike CPU operations constrained to 64-bit words, GPU kernels operate on arbitrary-length bit arrays stored in VRAM. This eliminates the "64-bit horizon" entirely for GPU-accelerated workloads.

Table 12 shows batch operation performance.

Operation	**CPU**	**GPU**	**Speedup**
Bulk AND (1K)	56 ns	180 ns	0.3×
Bulk AND (10K)	560 ns	220 ns	2.5×
Bulk AND (100K)	5.6 μs	280 ns	20×
Bool MatMul (128×128)	2.1 ms	45 μs	47×
Transitive Closure (64×64)	890 μs	18 μs	49×

Table 12: GPU vs CPU: Speedup at large batch sizes

Memory Transfer Bottleneck. The 0.3× slowdown for Bulk AND (1K) reflects PCIe transfer overhead dominating compute. GPU dispatch requires uploading data to VRAM, launching the kernel, and downloading results—a fixed cost of $\sim$100μs regardless of workload. For small batches, this overhead exceeds CPU execution time. The crossover point occurs around 10K operations; beyond that, GPU parallelism dominates and speedup grows with batch size.

Batch Sudoku. Solving 1000 random Sudoku puzzles:

- CPU sequential: 31 ms (31 μs/puzzle)
- GPU parallel: 1.2 ms (1.2 μs/puzzle)
- Speedup: 26×

10 New Results: Symbolic Addition (MNIST-Addition Concept)

We demonstrate the MNIST-Addition neurosymbolic concept with simplified digit patterns:

- **Task:** Given pairs of noisy digit patterns (a, b) and target sums c, learn a classifier that maps patterns to digits while satisfying the logical constraint $\text{classify}(a) + \text{classify}(b) = c$.
- **Architecture:** Linear classifier (10×10 weights) with softmax output, followed by differentiable addition constraint.

- **Training:** Temperature annealing from soft ($\tau = 2$) to hard ($\tau = 0.1$), cross-entropy loss on constraint satisfaction.

Key insight: The addition constraint

$$P(\text{sum} = c) = \sum_{d_1+d_2=c} P(a \to d_1) \cdot P(b \to d_2)$$

is fully differentiable via our semiring abstraction (soft-AND = product, soft-OR = sum over valid pairs).

Epoch	Loss	Accuracy
0	2.68	100%
20	2.48	100%
49	0.83	100%

Table 13: Symbolic addition learning: loss decreases, accuracy maintained

This demonstrates the core MNIST-Addition mechanism: gradients flow through both the neural classifier and the logical constraint, enabling end-to-end learning from pattern+sum supervision.

11 New Results: Differentiable Logic

Our `learn_chirho` module implements differentiable miniKanren:

- **LearnableRelationExtChirho**: Relations with learnable tuple weights
- **AnnealingScheduleChirho**: Temperature annealing (exponential, linear, cosine)
- **GumbelSoftmaxSamplerChirho**: Differentiable discrete choices
- **StraightThroughEstimatorChirho**: Hard forward pass, soft backward

Family Relations Demo. Learning parent relation from grandparent supervision:

- Given: grandparent(A, C) = parent(A, B) $\wedge$ parent(B, C)
- Supervision: grandparent facts only
- Result: Correctly learns parent weights from noisy candidates
- Convergence: 50 iterations, 0.01 learning rate

Differentiable Hierarchical Domains. We extend the 1-bit approach to soft probabilities with `DiffHierarchical4kChirho`:

- Each bit position has probability $p \in [0, 1]$ instead of hard 0/1
- Soft AND: $P(x \in A \cap B) = P(x \in A) \cdot P(x \in B)$
- Soft OR: $P(x \in A \cup B) = P(A) + P(B) - P(A) \cdot P(B)$
- Gradients flow through operations for end-to-end learning
- Temperature annealing sharpens distributions during training

Domain Type	Size	Intersect	Overhead
BitVec64Chirho	64	420 ps	1×
Hierarchical4kChirho	4,096	39 ns	93×
Hierarchical256kChirho	262,144	367 ns	870×
DiffHierarchical4kChirho	**4,096 soft**	**1.26 μs**	**3,000×**

Table 14: Domain composition benchmarks: hard vs soft operations

The soft version incurs 36× overhead vs hard Hierarchical4kChirho, but enables gradient-based learning through logic programs. GPU domains use `Vec<u32>` arrays and scale to unlimited size.

Training vs. Inference. The 3,000× overhead of differentiable domains applies only during *training*—when learning relation weights or neural-symbolic parameters. Once learned, the final model compiles back to hard 1-bit operations for inference. This mirrors the neural network paradigm: training is expensive (soft gradients), inference is cheap (hard forward pass). Our system supports both phases: `DiffHierarchical4kChirho` for training, then conversion to `Hierarchical4kChirho` for deployment.

12 New Results: Type Inference Case Study

To demonstrate the approach on a real-world application, we implement Hindley-Milner type inference as a miniKanren program. Type inference is a natural fit: types are terms, type checking is unification.

12.1 Implementation

Our implementation (`type_infer_chirho.rs`) encodes:

- **Base types**: `Int`, `Bool`, `String` as domain values 0, 1, 2
- **Function types**: Arrow types `a -> b` as term pairs

- **Type variables**: Fresh miniKanren variables
- **Constraints**: Unification goals expressing type rules

Type inference rules translate directly to goals:

- **Literals**: `42` : `Int`, `true` : `Bool`
- **Lambda**: $\lambda x.e : a \to b$ where $x : a$ and $e : b$
- **Application**: If $f : a \to b$ and $x : a$, then $(f\ x) : b$
- **If-then-else**: Condition must be `Bool`, branches must match

12.2 Performance

Expression	**Time**	**Notes**
Integer literal	0.8 μs	Single unification
Identity function ($\lambda x.x$)	2.1 μs	Polymorphic type
Application (id 42)	4.3 μs	Instantiation
If-then-else	6.7 μs	5 constraints
Type error detection	1.2 μs	Fast failure

Table 15: Type inference benchmarks (Rust)

The key insight: constraint propagation via bit-parallel operations enables fast type checking. Type errors manifest as domain intersection yielding $\vec{0}$ (no valid types), detected in a single cycle.

12.3 Scaling

For programs with n type constraints:

- 10 constraints: 15 μs
- 50 constraints: 89 μs
- 100 constraints: 234 μs

Linear scaling with constraint count, as expected for propagation-based inference.

13 Limitations

We acknowledge several limitations of the current approach:

1. Finite Domain Requirement. The bit-matrix representation requires domains to fit in fixed-width bit vectors. While hierarchical domains scale to 262K values and GPU domains are unbounded, truly infinite domains (e.g., all integers) require the hash-consing bridge, which adds overhead. For infinite recursive structures, tabling ensures termination but may not enumerate all solutions.

2. Symbolic Arithmetic. Arithmetic over bit-encoded integers requires explicit constraint encoding. Unlike SMT solvers with theory support, we cannot directly solve $x + y = z$ for arbitrary integers. We handle finite arithmetic (e.g., Sudoku digits 1-9) efficiently, but general arithmetic requires external integration.

3. GPU Transfer Overhead. GPU acceleration requires data transfer over PCIe. For small workloads (<10K operations), transfer overhead dominates compute time, resulting in slowdowns vs. CPU (see Table 12, 0.3× for 1K operations). GPU benefits emerge at batch sizes >10K. A fully fused GPU-resident implementation would eliminate this overhead.

4. Differentiable Performance Cliff. Soft-logic operations incur 3,000× overhead vs. hard 1-bit operations. This is acceptable for training (where gradient flow is essential) but prohibitive for inference. Users must compile trained models back to hard operations for deployment.

5. FPGA Hardware Status. Current FPGA results (8-cycle latency, ∼2000 LUT estimate) are from simulation (Verilator) and resource estimation. Actual synthesis on physical hardware (iCE40, Artix-7) is pending; measured resource counts and power consumption may differ from estimates. The constraint propagation engine is synthesizable; SLG tabling remains software-only.

6. Contraction Order Complexity. Optimal tensor contraction order is NP-hard (equivalent to treewidth computation). Our greedy/min-fill heuristics are typically within 2–10× of optimal, but pathological cases may exhibit exponential blowup.

14 Related Work

Logic programming implementations. SWI-Prolog, XSB [23], and μKanren [14] use traditional term representation with pointer-based substitutions. faster-miniKanren [4] optimizes stream handling for better performance. OCanren [16] provides typed embedding in OCaml. Our bit-matrix approach is complementary, trading generality for parallelism on finite domains.

Constraint solvers. SAT solvers (MiniSat [10], Z3 [9]) and ASP solvers

(clingo [13]) operate on Boolean/integer domains. Constraint Logic Programming over Finite Domains [12] uses similar constraint propagation. Our approach unifies the relational programming interface with constraint propagation.

E-graphs. egg [24] provides equality saturation via e-graphs, building on congruence closure [19]. Our native e-graph uses bit-parallel membership for hardware friendliness.

Differentiable logic. Scallop [17] provides differentiable Datalog with provenance semirings. DeepProbLog [18] integrates neural networks with probabilistic logic. Neural theorem provers [22] use differentiable unification. Gumbel-softmax [15] enables gradient flow through discrete choices. Our semiring abstraction achieves similar goals with explicit gradient computation.

Program synthesis. SyGuS [2] defines syntax-guided synthesis problems. CVC5 [5] is a state-of-the-art SyGuS solver. Barliman [7] uses miniKanren for program synthesis. Our domain-pruned enumeration achieves speedups on simple problems.

Hardware logic programming. Prior work on Prolog machines (Warren Abstract Machine [1]) focused on instruction-level parallelism. Our tensor approach targets data-level parallelism via SIMD/GPU/FPGA. Calyx [20] and Clash [3] enable verified hardware generation.

Tabling and memoization. SLG resolution [8] provides sound tabling for logic programs. XSB [23] implements tabling for Prolog. Our mode-driven goal reordering builds on these techniques.

Tensor networks. Tensor network theory [6] and tensor-train decomposition [21] provide the mathematical foundation for our representation.

Categorical logic. Our tensor contraction approach connects to string diagrams in monoidal categories [**?**]. Relations correspond to morphisms, variable sharing to wiring, and contraction order to coherence conditions. The Topos Institute's work on compositional systems [**?**] provides the theoretical foundation for viewing our approach categorically: tensor networks *are* string diagrams, and optimization over contraction order is diagram rewriting.

15 Conclusion

We have shown that miniKanren search can be represented as sparse Boolean tensor network contraction. This formulation enables:

- 3,000–4,000× speedup on constraint operations
- Direct mapping to FPGA/ASIC primitives
- Differentiable relaxation for neural-symbolic integration

The observation that unification over finite domains is bitwise AND enables parallelism while preserving the relational programming model.

Code Availability. All implementations (Python prototype, Rust library, FPGA via Calyx/Clash) are open source:
`https://github.com/loveJesus/minikanren_1bit_chirho`

The Rust implementation is available on crates.io as `minikanren_1bit_chirho`.

Future work. While we have demonstrated hierarchical domains up to 262K values and GPU acceleration, several directions remain: (1) Tighter neural-symbolic integration, including differentiable parsing and program induction; (2) FPGA deployment on production boards with full tabling support; (3) Application to larger synthesis benchmarks (SyGuS-Comp full suite); (4) *Fully fused logic accelerator*—current GPU implementation incurs PCIe transfer overhead per operation. A fused design would keep all data GPU/FPGA-resident: hash-consed term store in VRAM/BRAM, domain propagation as on-chip kernels, and only final solutions transferred back. This eliminates the memory hierarchy bottleneck that dominates small workloads (see Table 12, 0.3× slowdown for 1K operations). Early analysis suggests 10-100× improvement for deep search with many iterations.

Hardware Configuration

All benchmarks were run on:

- **CPU:** Apple M4 Pro (14 cores: 10 performance + 4 efficiency)
- **Memory:** 48 GB unified memory
- **OS:** macOS Darwin 25.2.0
- **Rust:** 1.70+ with release optimizations

Reproducibility for Frontier LLMs

This paper is designed to be reproducible by frontier language models. To reproduce all results:

```
# Clone the repository
git clone https://github.com/loveJesus/minikanren_1bit_chirho
cd minikanren_1bit_chirho/rust_chirho

# Run all tests (should report 209+ passing)
cargo test

# Run benchmarks
cargo bench --bench criterion_bench_chirho
```

```
# Run examples
cargo run --example sudoku_chirho
cargo run --example nqueens_chirho
cargo run --example type_infer_chirho
cargo run --example synthesis_chirho
cargo run --example profile_breakdown_chirho
cargo run --example learn_deep_chirho

# Parallel scaling (requires parallel feature)
cargo bench --features parallel_chirho --bench parallel_bench_chirho
```

LLM Reproduction Prompt: "Read AGENTS.md and paper_chirho.tex from the minikanren_1bit_chirho repository. Implement the core mapping (unification = bitwise AND, conde = row duplication) in your preferred language. Verify by solving 8-Queens and comparing performance against a stream-based miniKanren."

Acknowledgments

Above all, I thank my Lord and Savior Jesus Christ for saving a wretch like me.

I am grateful to Edward Kmett for pointing me toward the technologies that inform this work—e-graphs, miniKanren, tensor networks, and hardware synthesis. His learning path provided the roadmap that made this exploration possible.

I also thank my father, Roger Cowan, for his patient support in working through ideas and helping me think clearly about the problem space.

This paper was developed with assistance from Claude (Anthropic) and Google Gemini, which contributed to structure, refinement, and feedback on earlier drafts.

Finally, I thank the countless researchers and engineers who built the foundations we stand on: the miniKanren community, the egg/e-graph developers, the Rust ecosystem, the Calyx and Clash teams, and the broader logic programming and tensor network communities.

References

[1] Hassan Aït-Kaci. *Warren's Abstract Machine: A Tutorial Reconstruction*. MIT Press, 1991.

[2] Rajeev Alur et al. Syntax-guided synthesis. In *FMCAD*, 2013.

[3] Christiaan Baaij et al. CaSH: Structural descriptions of synchronous hardware using Haskell. *EUROMICRO DSD*, 2010.

[4] Michael Ballantyne, Gregory Rosenblatt, and William E. Byrd. faster-minikanren: Making miniKanren run faster. In *miniKanren Workshop*, 2017.

[5] Haniel Barbosa et al. cvc5: A versatile and industrial-strength SMT solver. In *TACAS*, 2022.

[6] Jacob C. Bridgeman and Christopher T. Chubb. Hand-waving and interpretive dance: An introductory course on tensor networks. *Journal of Physics A: Mathematical and Theoretical*, 50(22), 2017.

[7] William E. Byrd, Michael Ballantyne, Gregory Rosenblatt, and Matthew Might. A unified approach to solving seven programming problems (functional pearl). In *ICFP*, 2017.

[8] Weidong Chen and David S. Warren. Tabled evaluation with delaying for general logic programs. *Journal of the ACM*, 43(1):20–74, 1996.

[9] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *TACAS*, 2008.

[10] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *SAT*, 2003.

[11] Daniel P. Friedman, William E. Byrd, and Oleg Kiselyov. *The Reasoned Schemer*. MIT Press, 1st edition, 2005.

[12] Thom Frühwirth and Slim Abdennadher. *Essentials of Constraint Programming*. Springer, 2003.

[13] Martin Gebser et al. clingo = ASP + control: Preliminary report. In *Technical Communications of ICLP*, 2014.

[14] Jason Hemann and Daniel P. Friedman. μKanren: A minimal functional core for relational programming. In *Scheme and Functional Programming Workshop*, 2013.

[15] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. In *ICLR*, 2017.

[16] Dmitry Kosarev and Dmitry Boulytchev. OCanren: Typed embedding of a relational language in OCaml. *Electronic Proceedings in Theoretical Computer Science*, 285:1–22, 2018.

[17] Ziyang Li et al. Scallop: A language for neurosymbolic programming. In *PLDI*, 2023.

[18] Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *NeurIPS*, 2018.

[19] Greg Nelson and Derek C. Oppen. Fast decision procedures based on congruence closure. *Journal of the ACM*, 27(2):356–364, 1980.

[20] Rachit Nigam et al. A compiler infrastructure for accelerator generators. In *ASPLOS*, 2021.

[21] Ivan V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.

[22] Tim Rocktäschel and Sebastian Riedel. End-to-end differentiable proving. In *NeurIPS*, 2017.

[23] Terrance Swift and David S. Warren. XSB: Extending Prolog with tabled logic programming. In *Theory and Practice of Logic Programming*, 2012.

[24] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. egg: Fast and extensible equality saturation. In *POPL*, 2021.

Soli Deo Gloria $\chi\rho$